

①

Resumo 3

Cap 1 - Construindo software que faz diferença

Introdução:

- Problema recorrente na indústria: muitos projetos falham ou entregam pouco valor (ex: ECSS da força aérea do EUA cancelado após ~ US\$ 1 bi; relatórios CHAOS com altas taxas de atraso/cancelamento). BDD é apresentado como abordagem colaborativa para focar no que realmente importa, usando exemplos concretos e testes de aceitação automatizados

1.1 → BDD "a 50 mil pés":

- Processo Tradicional (Tipo "Chius"): Vários "Produtores" (analista) → (dev) → (tester) → (Técnico de docs) geram perdas de informação e mal-entendidos

• Processo com BDD (Tipo "Sarah"):

- ↳ Conversa conjunta (negócio + dev + QA) sobre objetivos e exemplos/contraexemplos
- ↳ Exemplos escritos em linguagem de negócio, estruturados
- ↳ Vivam testes de aceitação automatizados que guiam o dev.
- ↳ Testes dizem quando "terminou" e rodam de base para testes exploratórios
- ↳ Relatórios dos testes viram documentação viva do projeto

spiral

tamanho subvaga

__/__/__
(2)

S T Q Q S S D

1.2 Que problemas você está tentando resolver?

- Dois eixos do sucesso: construir de jeito certo (qualidade interna) e construir a coisa certa (alinhamento ao negócio)

↳ Falhar em qualquer um leva a software caro, frágil ou inútil

1.2.1 → Construindo de ~~jeito~~ certo:

- Baixa qualidade interna resulta em bugs, lentidão para mudar, documentação desatualizada e longos ciclos manuais de teste

- Praticar como TDD, clean code, CI e testes automatizados costumam correlacionar com menos defeitos e código mais adaptável (não é bala de prata, mas ajuda).

1.2.2 → Construindo a coisa certa:

- Software precisa alinhar com objetivos de negócio; caso contrário, entrega funcionalidades pouco usadas (estudos indicam ~45% de features nunca utilizadas)

- Exemplo de consequência: projeto de folha de pagamento de Queensland (AU) → Atraso grande, pagamentos incorretos e custos enormes para correção.

- BDD ajuda a reduzir esse risco ao esclarecer metas e validar valor cedo, por meio de exemplos que visam critérios de aceitação.

③

1.2.3 → A restrição do conhecimento (lida com incerteza)

• Incerteza é inevitável; o entendimento cresce ao longo do projeto e não de forma linear. O trabalho do time é descobrir o que construir e como.

• Em vez de "travar" requisitos cedo, gerencie a incerteza para ir convergindo ao que traz valor; "quando o terreno diverge do mapa, confie no terreno".

1.3 → BDD serve para seu projeto?

• BDD não substitui SCRUM/XP/KANBAN; complementa.

↳ EX: aplicar em refinement para clarear histórias, buscar automação in-sprint e incluir "texto de aceitação" na Definição of done.

• Funciona em greenfield e legados; é útil em domínio não trivial e incerto (praticamente a maioria)

1.4 → O que você vai aprender no livro:

• Descobrir requisitos reais colaborativamente

• Registrar requisitos como especificações executáveis e integrá-las ao build.

• Automatizar essas especificações

• Gerar documentação viva que verifica e documenta o comportamento entregue

spiral6